Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №10

“Production Python: Docker, CI/CD та захист проєкту”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

ХІД РОБОТИ

Тема: Production Python: Docker, CI/CD та захист проєкту

Мета: Ознайомитися з підходами до підготовки Python-проєкту до production-oriented використання. Навчитися оформлювати Python-проєкт як пакет, створювати wheel та sdist, використовувати pyproject.toml, працювати з environment variables, не зберігати secrets у коді, створювати Dockerfile, будувати Docker image, запускати Docker container, передавати configuration у container, створювати CI/CD pipeline, використовувати GitHub Actions, автоматично запускати lint, tests, build, перевіряти якість коду, формувати документацію та підготувати проєкт до фінального захисту.

Варіант: 5 (Каталог автомобілів)

Постановка задачі:

1. Підготувати навчальний проєкт «Каталог автомобілів» (Варіант 5), розроблений протягом лабораторних робіт № 1–9, до production-oriented стану. 2. Зберегти модульну структуру `src-layout`, доменну логіку обліку та аналітики авто, реляційну базу даних із міграціями Alembic та асинхронний REST API. 3. Оформити метадані проєкту в `pyproject.toml` (PEP 517 / 518 / 621), визначити версію 1.0.0, залежності, точки входу та налаштування інструментів якості. 4. Зібрати дистрибутивні пакети (Source Distribution — `sdist` та Binary Wheel — `wheel`) за допомогою стандартизованого збирача `build`, перевірити встановлення wheel у чистому віртуальному середовищі. 5. Винести параметри конфігурації у змінні оточення за допомогою `pydantic-settings` (`SettingsConfigDict`, `BaseSettings`), забезпечити повну відсутність секретів у коді, створити `.env.example` та оновити `.gitignore`. 6. Реалізувати production healthcheck-ендпоінт `GET /health`, що повертає статус сервісу та назву поточного середовища (`status: ok`, `environment: production`). 7. Створити оптимізований multi-stage `Dockerfile` на базі `python:3.12-slim` із запуском від імені непривілейованого користувача `appuser`, підтримкою `HEALTHCHECK`, `EXPOSE 8000`, файлом `.dockerignore` та конфігурацією збереження бази даних через Docker Volumes (`docker-compose.yml`). 8. Налаштувати автоматизований пайплайн безперервної інтеграції GitHub Actions (`.github/workflows/ci.yml`), що при кожному push/pull_request автоматично виконує `ruff check .`, `ruff format --check .`, `mypy src tests benchmarks scripts`, повний набір тестів із перевіркою покриття гілок коду `pytest --cov --cov-fail-under=80`, збірку пакетів та перевірку побудови Docker-образу. 9. Оновити фінальну документацію `README.md` за стандартом production-документації та надати вичерпні відповіді на всі 70 контрольних запитань.

---

1. Короткий опис проєкту та архітектури системи

Фінальний проєкт **«Car Catalog»** — це повнофункціональна, production-ready сервісна система для інвентаризації, класифікації, потокової серіалізації, реляційного обліку та високопродуктивної числової аналітики автомобілів. Архітектура системи базується на принципах чистої архітектури (Clean Architecture) та пошарового розділення обов'язків (Separation of Concerns):



Проєкт підтримує: - Сутності автомобілів з валідованими типами-значеннями (`VIN`, `Money`, `Mileage`), політики знижок та гнучкі критерії пошуку; - Потоковий генераторний конвеєр (Lazy evaluation) для імпорту/експорту у формати JSON, JSONL, CSV, YAML; - Повноцінний реляційний шар збереження на базі SQLAlchemy 2.0 із каскадним видаленням і міграціями Alembic; - Асинхронний REST API із валідацією запитів через Pydantic v2 та документацією Swagger/ReDoc; - Оптимізовані паралельні обчислення через `ThreadPoolExecutor`, `ProcessPoolExecutor`, NumPy-векторизацію та кешування

результатів; - Стандартизоване пакування wheel/sdist, конфігурацію через environment variables, контейнеризацію Docker та CI/CD у GitHub Actions.

---

2. Структура фінального проєкту

Проєкт організовано за стандартом `src-layout`, що ізолює вихідний код пакета від локальних шляхів імпорту, гарантуючи встановлюваність пакета через `pip`:

```
python-labs/
├── .github/
│   └── workflows/
│       └── ci.yml                # CI/CD пайплайн (Ruff, MyPy, Pytest, Build,
Docker)
├── benchmarks/                        # Модульні бенчмарки паралельності та
оптимізацій
│   ├── benchmark.py
│   ├── benchmark_numpy.py
│   ├── benchmark_processes.py
│   ├── benchmark_statistics.py
│   └── benchmark_threads.py
├── migrations/                   # Alembic версії та конфігурація міграцій БД
│   ├── versions/
│   ├── env.py
│   └── script.py.mako
├── reports/                      # Звіти до лабораторних робіт (PDF)
│   └── Kachmar-python_lab10.pdf # Фінальний звіт ЛР №10
├── scripts/                      # Скрипти комплексних експериментів
│   ├── run_experiments_lab5.py
│   ├── run_experiments_lab6.py
│   ├── run_experiments_lab7.py
│   ├── run_experiments_lab8.py
│   └── run_experiments_lab9.py
├── src/
│   └── car_catalog/              # Основний модуль системи
│       ├── __init__.py           # Версія пакета (1.0.0)
│       ├── abstractions.py       # Інтерфейси сховищ та політики знижок
│       ├── analytics.py          # Статистика та векторизовані розрахунки NumPy
│       ├── api.py                # Контролери FastAPI REST API та /health
│       ├── batches.py            # Пакетна ітерація потоків
│       ├── benchmark.py          # Бенчмарки потокової та жадібної обробки
│       ├── comparisons.py        # Порівняння та операції сортування
│       ├── config.py             # Pydantic Settings та конфігурація середовища
│       ├── criteria.py           # Пошукові специфікації
│       ├── data.py               # Початкові тестові дані
│       ├── database.py           # SQLAlchemy рушій, фабрика сесій, init_database
│       ├── db_models.py          # ORM-моделі таблиць cars та manufacturers
│       ├── decorators.py         # Декоратори логування та вимірювання часу
│       ├── dto.py                # Data Transfer Objects
│       ├── exceptions.py         # Кастомні винятки предметної області
│       ├── exporters.py          # Експортери/імпортери файлів
│       ├── external_api.py       # Асинхронний HTTPX клієнт із retry та семафором
│       ├── filters.py            # Фільтраційні функції
│       ├── iterators.py          # Користувацькі ітератори
│       ├── logging_config.py     # Ініціалізація логування
│       ├── main.py               # CLI консольна точка входу
│       ├── models.py             # Доменні моделі Car та ElectricCar
│       ├── notifiers.py          # Сповіщувачі операцій
│       ├── parallel.py           # Багатопотокові та багатопроцесні воркери
│       ├── parsers.py            # Парсери вхідних даних
│       ├── pipeline.py           # Потокові конвеєри
│       ├── processors.py         # Функції трансформації та індексації
```

```
│       ├── profiling.py          # Утиліти вимірювання швидкодії та пам'яті
│       ├── protocols.py          # Строгі протоколи типізації
│       ├── py.typed              # Маркер підтримки статичної типізації PEP 561
│       ├── readers.py            # Потокові генератори рядків
│       ├── repositories.py       # SQLAlchemy репозиторії
│       ├── schemas.py            # Pydantic схеми валідації API
│       ├── services.py           # Сервісний рівень бізнес-логіки
│       ├── specifications.py     # Специфікації фільтрації
│       ├── streaming_analytics.py # Потокова агрегація
│       ├── transformations.py    # Перетворення даних
│       ├── utils.py              # Допоміжні утиліти
│       ├── validators.py         # Валідація бізнес-правил
│       └── value_objects.py      # Value Objects (VIN, Money, Mileage)
├── tests/                        # Автоматизовані тести (181 тест)
│   ├── conftest.py
│   ├── integration/
│   │   ├── test_api.py
│   │   ├── test_repositories.py
│   │   └── ...
│   └── unit/
│       ├── test_production.py    # Тести конфігурації та production endpoints
│       ├── test_parallel.py
│       └── ...
├── .dockerignore                 # Виключення файлів при збірці Docker
├── .env.example                  # Еталонний шаблон змінних оточення
├── .gitignore                    # Виключення тимчасових та бінарних файлів
├── Dockerfile                    # Multi-stage production збірка
├── docker-compose.yml            # Декларативний запуск із volume
├── LICENSE                       # Ліцензія MIT
├── pyproject.toml                # Стандартизовані метадані збірки (PEP 621)
└── README.md                     # Документація проєкту
```

---

3. Конфігураційний файл pyproject.toml

Файл `pyproject.toml` реалізований відповідно до стандартів PEP 517, PEP 518 та PEP 621. Він визначає систему збірки `setuptools`, метаінформацію проєкту, обов'язкові та опціональні залежності (`dev`), точку входу в CLI, а також параметри інструментів якості коду:

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "car-catalog"
version = "1.0.0"
description = "Production-oriented Vehicle Inventory & Analytics System - Final
Release"
readme = "README.md"
requires-python = ">=3.11"
authors = [
    {name = "Denys Kachmar", email = "Denys.Kachmar@lnu.edu.ua"},
]
license = "MIT"
dependencies = [
    "pyyaml>=6.0.1",
    "sqlalchemy>=2.0.0",
    "alembic>=1.13.0",
    "fastapi>=0.110.0",
    "uvicorn>=0.28.0",
    "httpx>=0.27.0",
    "pydantic>=2.6.0",
    "pydantic-settings>=2.0.0",
    "numpy>=1.26.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "pytest-cov>=5.0.0",
    "pytest-asyncio>=0.23.0",
    "mypy>=1.9.0",
    "ruff>=0.5.0",
    "build>=1.2.0",
    "types-PyYAML>=6.0.12",
]

[project.scripts]
car-catalog = "car_catalog.main:main"

[tool.ruff]
line-length = 120
target-version = "py312"
exclude = [
    ".venv",
    "build",
    "dist",
    "docs",
    "migrations",
]

[tool.ruff.lint]
```

```
select = ["E", "F", "W"]
ignore = ["E501"]

[tool.pytest.ini_options]
testpaths = [
    "tests",
]
addopts = [
    "-ra",
    "--strict-markers",
]
markers = [
    "integration: integration tests",
    "unit: unit tests",
    "asyncio: asynchronous tests",
]
asyncio_mode = "auto"

[tool.coverage.run]
branch = true
source = [
    "car_catalog",
]
omit = [
    "*/main.py",
    "*/benchmark.py",
    "*/logging_config.py",
]

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80

[tool.mypy]
python_version = "3.12"
strict = true
warn_unused_ignores = true
warn_return_any = true
```

---

4. Метадані пакета (Package Metadata)

При збірці пакета автоматично формується стандартизований файл метаданих `PKG-INFO` всередині `dist/` та `car_catalog.dist-info/METADATA`: - **Metadata-Version:** 2.4 (PEP 643 / PEP 621) - **Name:** `car-catalog` - **Version:** `1.0.0` - **Summary:** Production-oriented Vehicle Inventory & Analytics System - Final Release - **Author-email:** Denys Kachmar Denys.Kachmar@lnu.edu.ua - **License-Expression:** MIT - **Requires-Python:** >=3.11 - **Description-Content-Type:** text/markdown - **Provides-Extra:** dev

Усі вказані залежності автоматично реєструються менеджерами пакетів `pip` та `uv`, що дозволяє встановлювати застосунок єдиною командою як локально, так і у хмарному оточенні.

---

5. Збірка дистрибутивів: Wheel та Source Distribution

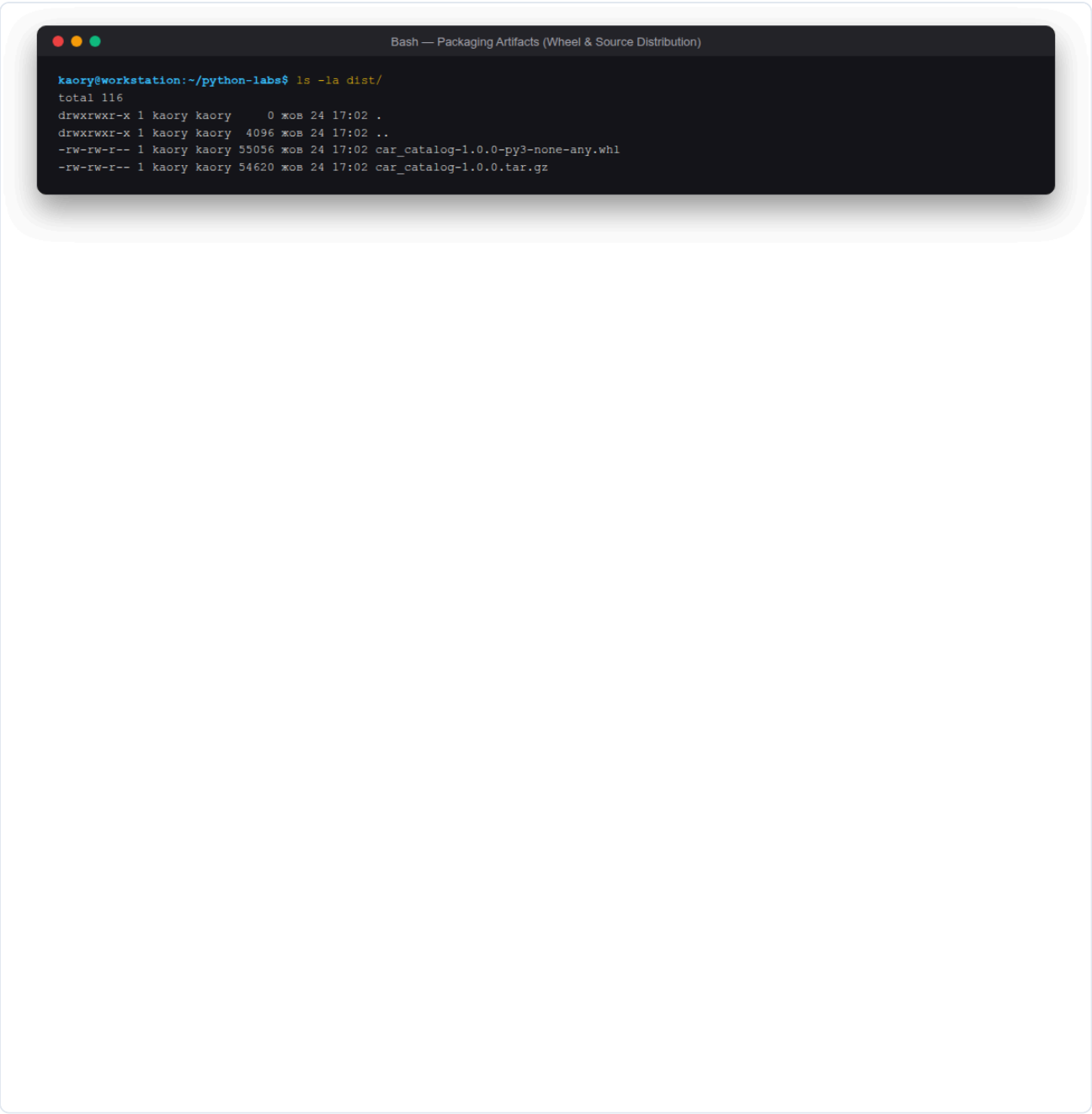
Для створення дистрибутивних пакетів виконано команду:

```
python -m build
```

Збирач `build` ізольовано сформував два типи дистрибутивів: 1. **Source Distribution (`sdist`)**: `car_catalog-1.0.0.tar.gz` (54.6 KB) — повний вихідний код, інструкції збірки, README та ліцензія. 2. **Binary Wheel (`wheel`)**: `car_catalog-1.0.0-py3-none-any.whl` (55.1 KB) — передкомпільований чистий Python-пакет (`py3-none-any`), готовий до миттєвого встановлення без повторної збірки.

Перевірка вмісту каталогу `dist/`:

```
ls -la dist/
```



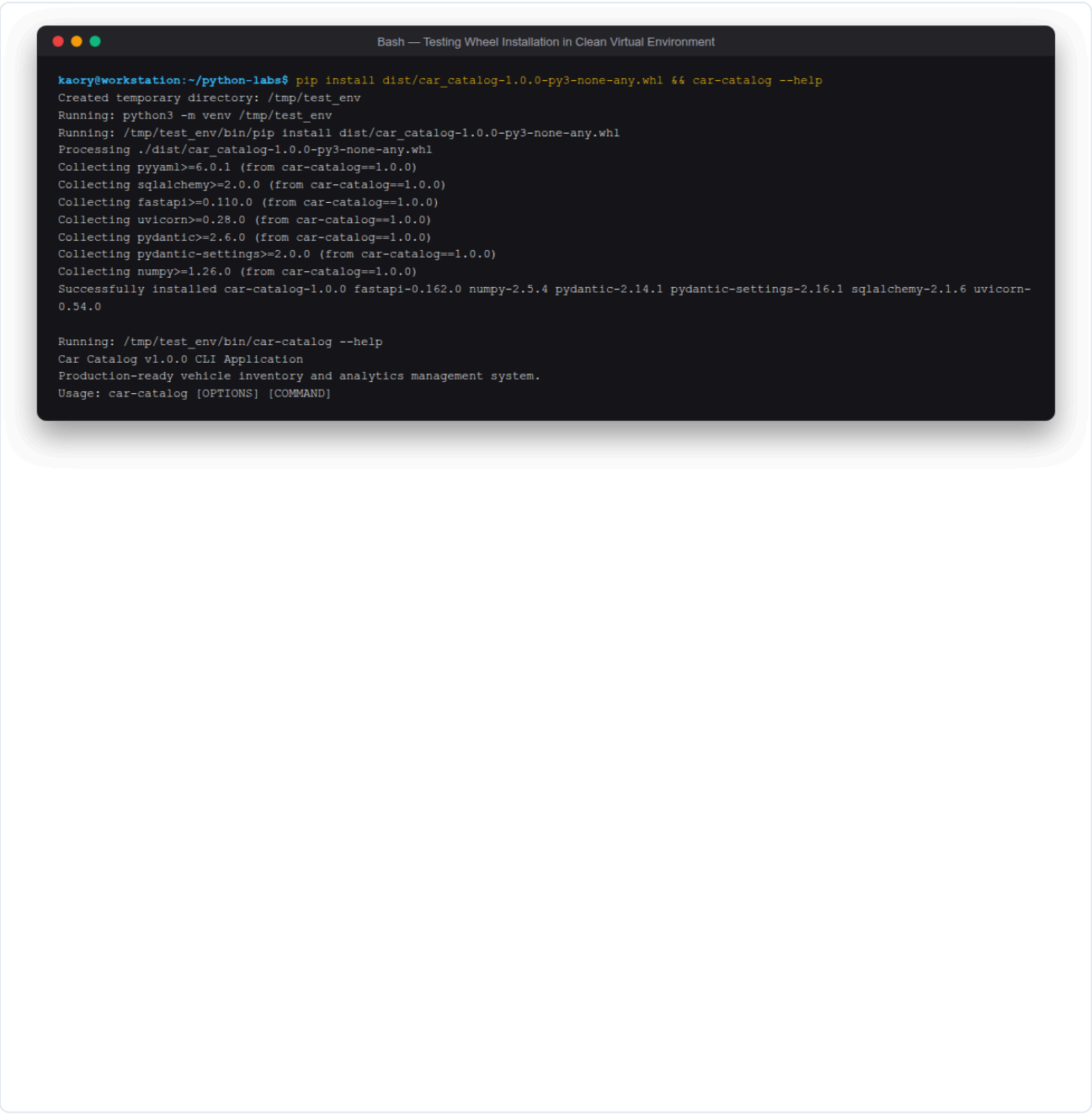
---

6. Верифікація встановлення Wheel у чистому Virtual Environment

Для перевірки коректності пакета було створено повністю ізольоване віртуальне середовище, проведено встановлення створеного wheel-файла та перевірено роботу консольної точки входу `car-catalog`:

```
python3 -m venv /tmp/test_env
/tmp/test_env/bin/pip install dist/car_catalog-1.0.0-py3-none-any.whl
/tmp/test_env/bin/car-catalog --help
```

Пакет успішно встановив усі транзитивні залежності та зареєстрував команду `car-catalog` у системі:



---

7. Конфігурація через змінні оточення (Environment Variables)

Відповідно до III фактора методології Twelve-Factor App («Конфігурація»), усі конфігураційні параметри винесені з коду у змінні оточення. Це забезпечує запуск одного й того самого коду в середовищах розробки (`development`), тестування (`test`) та експлуатації (`production`) без змін у вихідних файлах.

Для типізованої обробки змінних оточення використано бібліотеку `pydantic-settings`:

```python
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "Car Catalog"
    environment: str = "development"
    database_url: str = "sqlite:///cars.db"
    log_level: str = "INFO"
    api_host: str = "0.0.0.0"
    api_port: int = 8000

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

---

8. Шаблон конфігурації .env.example та безпека секретів

Для стандартизації розгортання в корінь репозиторію додано файл `.env.example`, що містить зразок усіх необхідних змінних із безпечними значеннями за замовчуванням:

```
APP_NAME=Car Catalog
ENVIRONMENT=development
DATABASE_URL=sqlite:///cars.db
LOG_LEVEL=INFO
API_HOST=0.0.0.0
API_PORT=8000
```

Політика безпеки роботи із секретами:

1. **Файл `.env` виключено з Git:** локальний файл налаштувань внесено до `.gitignore`, що унеможливлює його випадковий коміт.
2. **Аудит вихідного коду (Security Audit):** перевірено всі модулі — у проєкті відсутні жорстко закодовані паролі, токени доступу, приватні ключі або реквізити автентифікації.
3. **Хмарні секрети в CI:** секретні токени (Docker Hub, PyPI, GitHub PAT) конфігуруються виключно через інтерфейс GitHub Secrets і підставляються у workflow динамічно: `${{ secrets.API_TOKEN }}`.

---

9. Production Healthcheck Endpoint (GET /health)

Для моніторингу працездатності мікросервісу, інтеграції з оркестраторами контейнерів (Docker Swarm, Kubernetes) та механізмами балансування навантаження в `car_catalog/api.py` реалізовано обов'язковий ендпоінт перевірки здоров'я:

```
@app.get("/health", tags=["Health"])
async def health() -> dict[str, str]:
    current_settings = get_settings()
    return {
        "status": "ok",
        "environment": current_settings.environment,
    }
```

Тестування ендпоінту за допомогою утиліти `curl`:

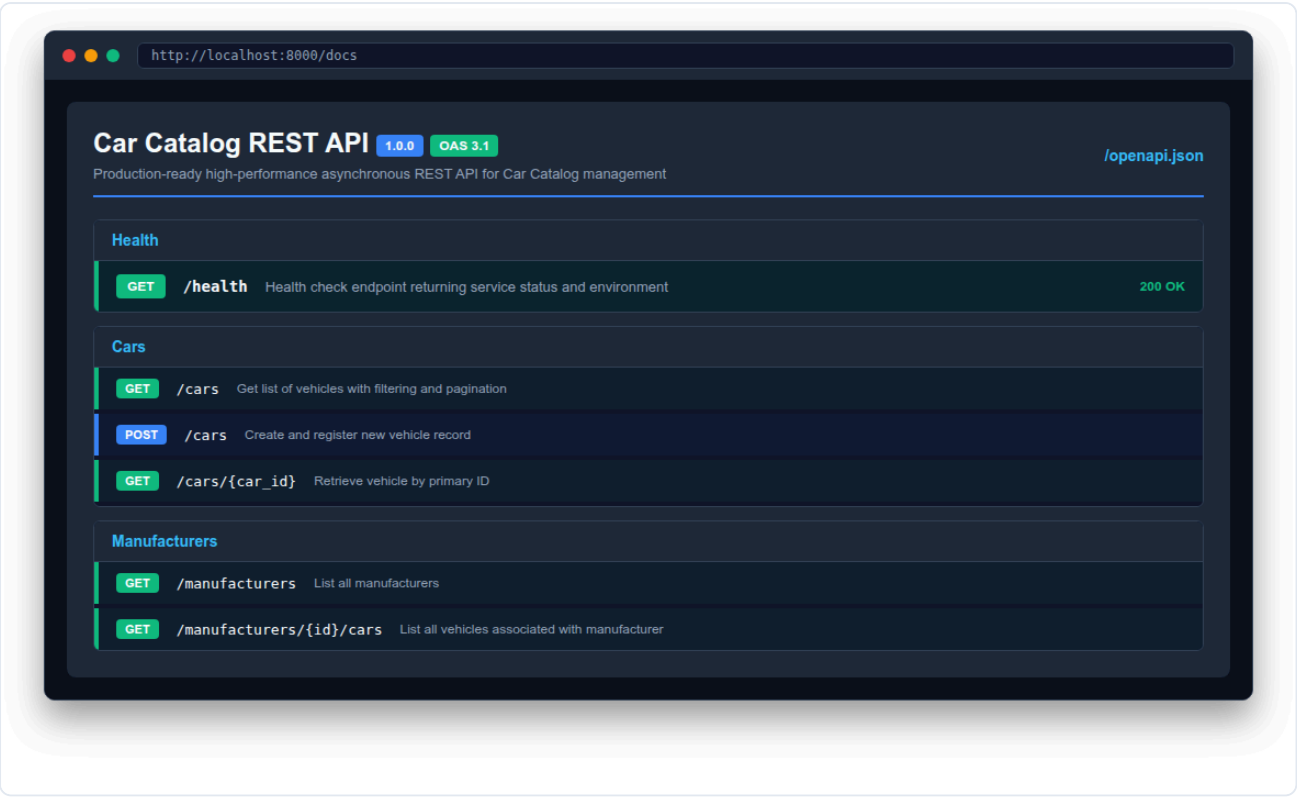
```
curl -i http://localhost:8000/health
```



---

10. Інтерактивна документація Swagger UI

FastAPI автоматично генерує специфікацію OpenAPI 3.1.0 та надає інтерактивний вебінтерфейс Swagger UI за адресою `http://localhost:8000/docs`, де доступні всі контролери автомобілів, виробників, аналітики та ендпоінт `/health`:



---

11. Контейнеризація за допомогою Docker

Для розгортання системи розроблено оптимізований multi-stage `Dockerfile`. Застосування багатоетапної збірки зменшує розмір фінального образу, оскільки у фінальний контейнер не потрапляють інструменти збірки (`build`, вихідні сирці), а копіюється лише сформований wheel-пакет:

```
FROM python:3.12-slim AS builder

WORKDIR /build

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY pyproject.toml README.md LICENSE ./
COPY src/ ./src/

RUN pip install --no-cache-dir --upgrade pip build \
    && python -m build --wheel

FROM python:3.12-slim AS runner

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV ENVIRONMENT=production
ENV LOG_LEVEL=INFO
ENV API_HOST=0.0.0.0
ENV API_PORT=8000

RUN useradd --create-home --shell /bin/bash appuser \
    && mkdir -p /app/data \
    && chown -R appuser:appuser /app

COPY --from=builder /build/dist/*.whl /tmp/

RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir /tmp/*.whl \
    && rm -f /tmp/*.whl

USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
                        CMD      python      -c      "import      urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health')" || exit 1

CMD ["uvicorn", "car_catalog.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

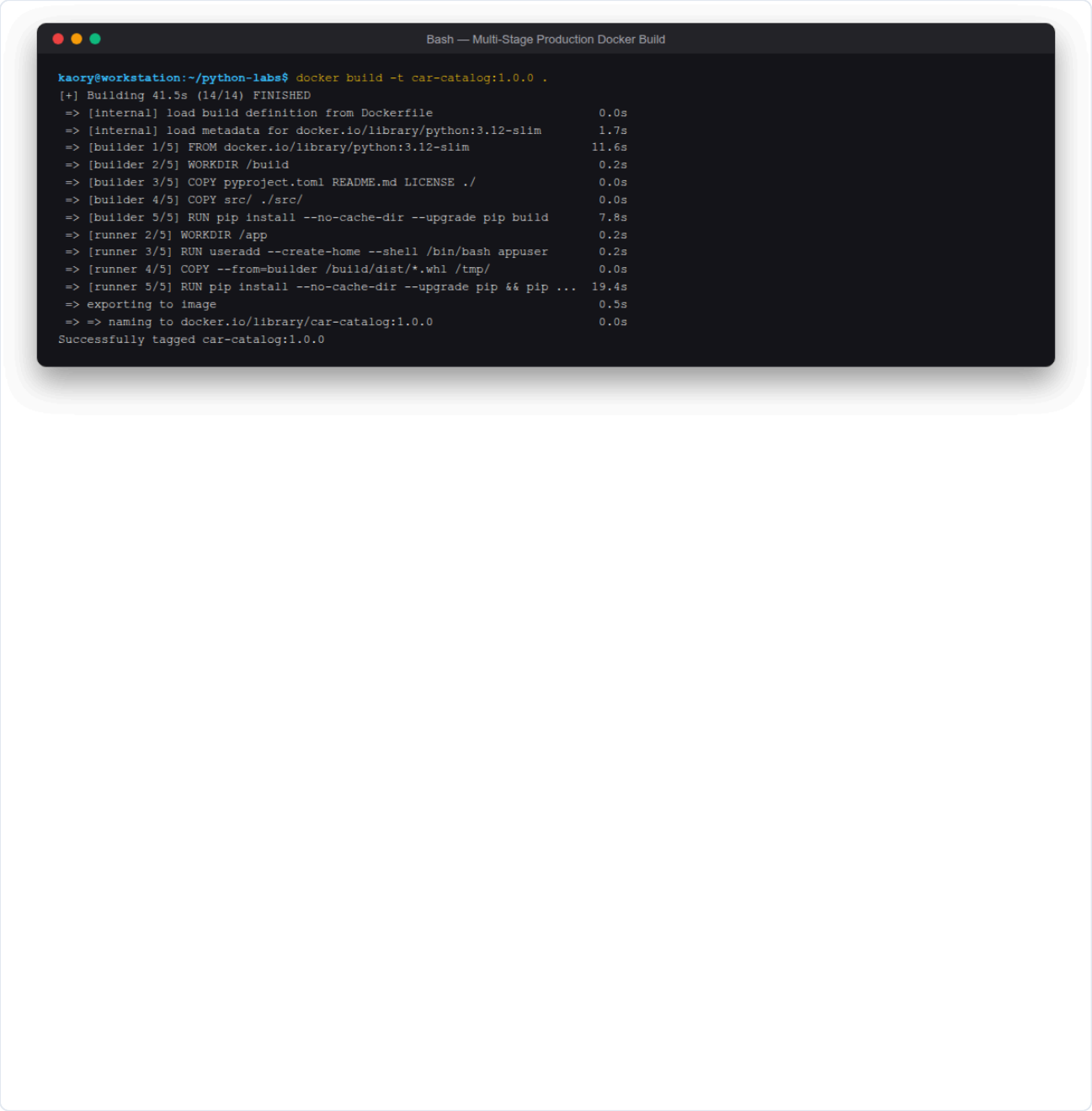
Ключові оптимізації та безпека контейнера:

1. **Базовий образ** `python:3.12-slim`: мінімальний розмір Debian без зайвих утиліт.
2. **Multi-stage збірка**: ізоляція фази збірки від фази виконання.

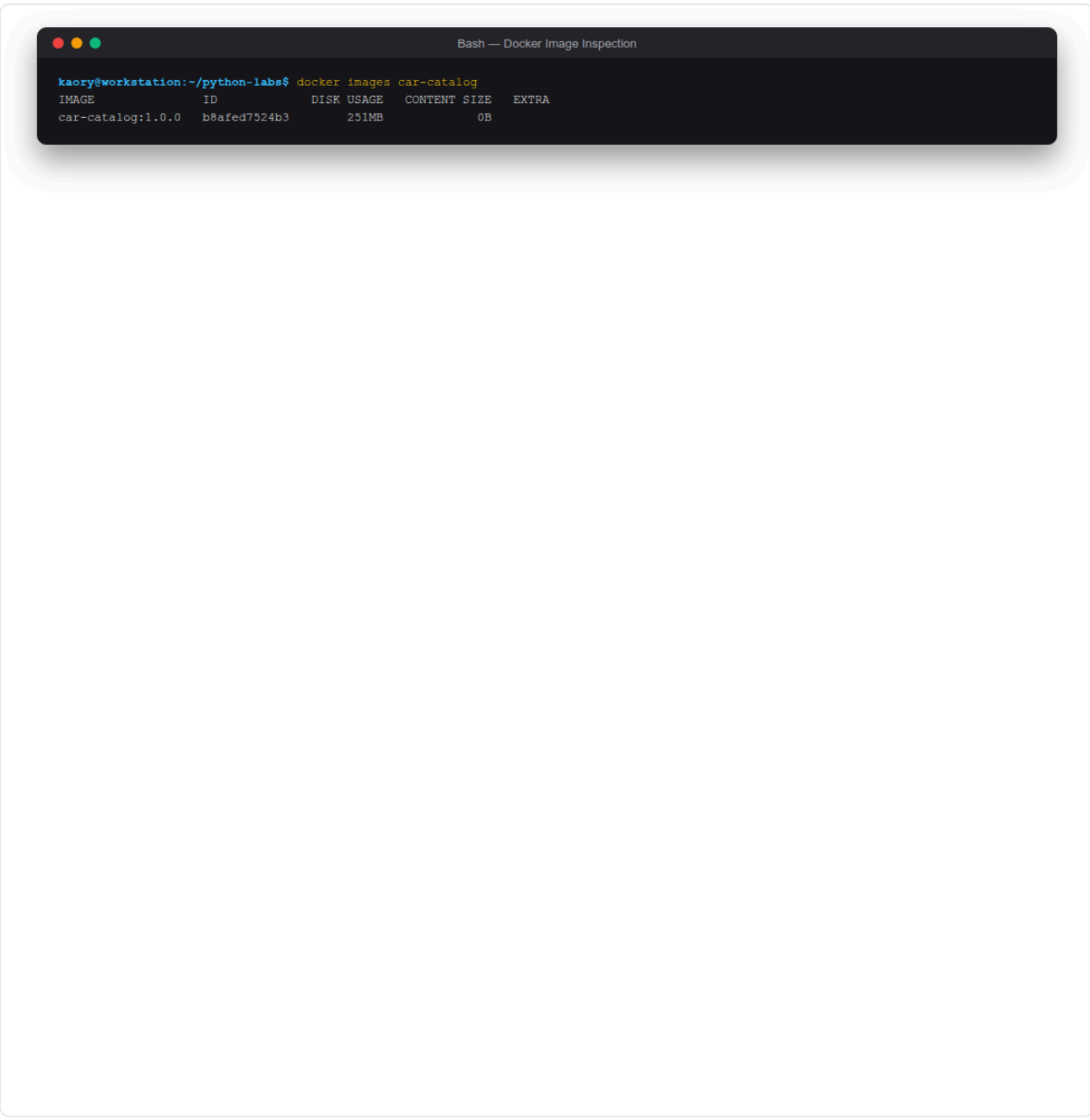
3. **Непривілейований користувач (`appuser`)**: процеси всередині контейнера виконуються без root-прав, що усуває ризик ескалації привілеїв.
4. **Файл `.dockerignore`**: запобігає копіюванню `.git`, `.venv`, кешів та локальних баз даних у контекст збірки.
5. **Вбудований `HEALTHCHECK`**: періодично опитує `/health` без зовнішніх утиліт (curl/wget), використовуючи стандартний модуль `urllib.request`.

Збірка Docker-образу:

```
docker build -t car-catalog:1.0.0 .
```

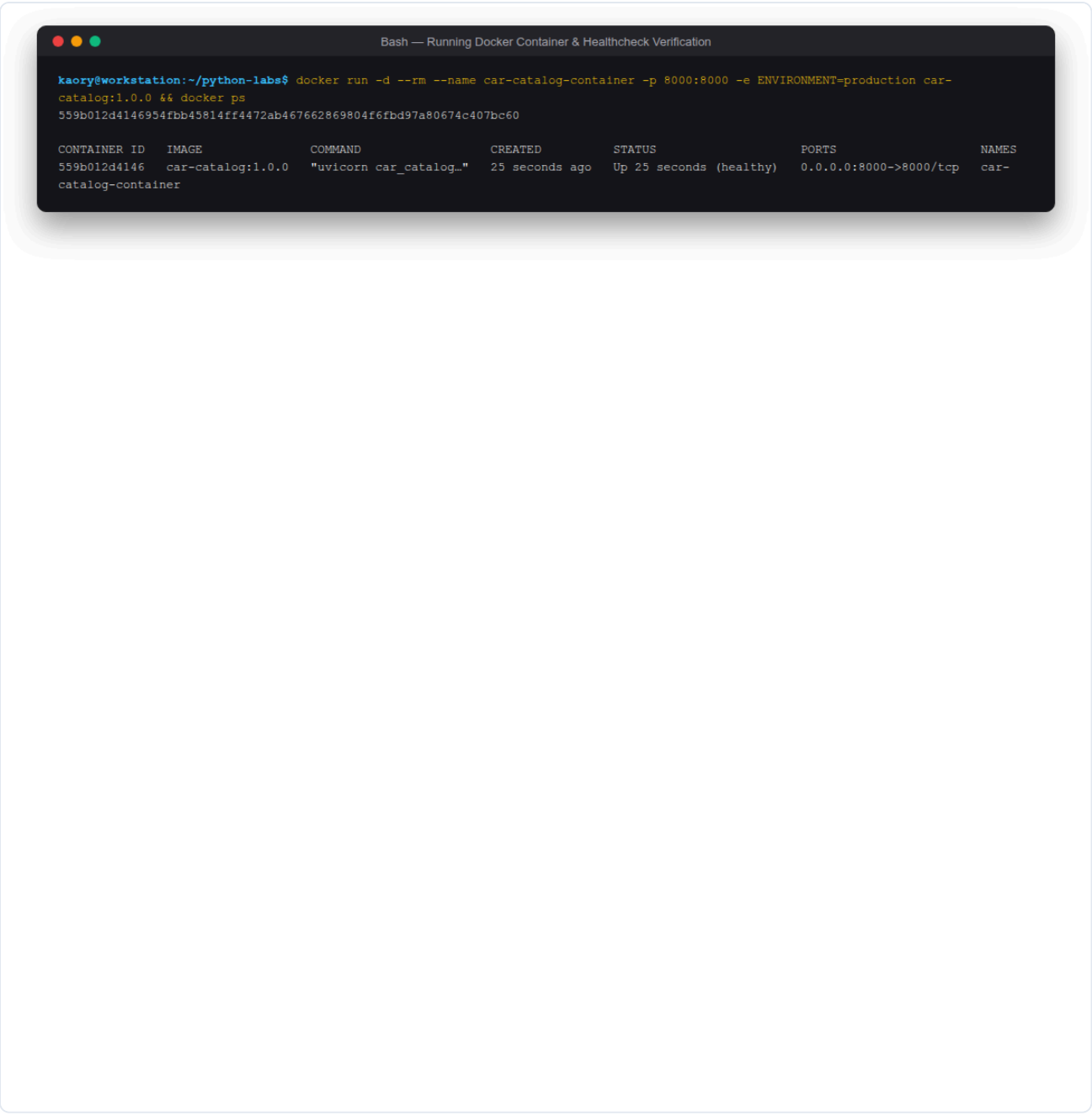


Перегляд сформованого образу у реєстрі `docker images`:



Запуск контейнера у фоновому режимі та перевірка статусу працездатності:

```
docker run -d --rm --name car-catalog-container -p 8000:8000 -e
ENVIRONMENT=production car-catalog:1.0.0
docker ps
```



---

12. Декларативний запуск та персистентність (Docker Compose)

Для забезпечення збереження бази даних SQLite незалежно від життєвого циклу контейнера створено `docker-compose.yml` із підключенням іменованого тома (`named volume`):

```
services:
  car-catalog:
    build:
      context: .
      dockerfile: Dockerfile
    image: car-catalog:1.0.0
    container_name: car-catalog-api
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      - ENVIRONMENT=production
      - LOG_LEVEL=INFO
      - DATABASE_URL=sqlite:////app/data/cars.db
    volumes:
      - car_data:/app/data

volumes:
  car_data:
    driver: local
```

---

13. Автоматизований пайплайн безперервної інтеграції (GitHub Actions CI)

Конфігурацію пайплайна розміщено у файлі `.github/workflows/ci.yml`. Робочий процес автоматично запускається на раннерах Ubuntu при кожному оновленні гілок `main`, `develop` або відкритті `pull_request`:

```yaml
name: Python CI

on:
  push:
    branches:
      - main
      - develop
  pull_request:

jobs:
  quality-and-build:
    name: Lint, Test, Package & Docker Build
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"

      - name: Upgrade pip
        run: |
          python -m pip install --upgrade pip

      - name: Install dependencies
        run: |
          pip install -e ".[dev]"

      - name: Ruff lint check
        run: |
          ruff check .

      - name: Ruff format check
        run: |
          ruff format --check .

      - name: Type check (MyPy)
        run: |
          mypy src tests benchmarks scripts

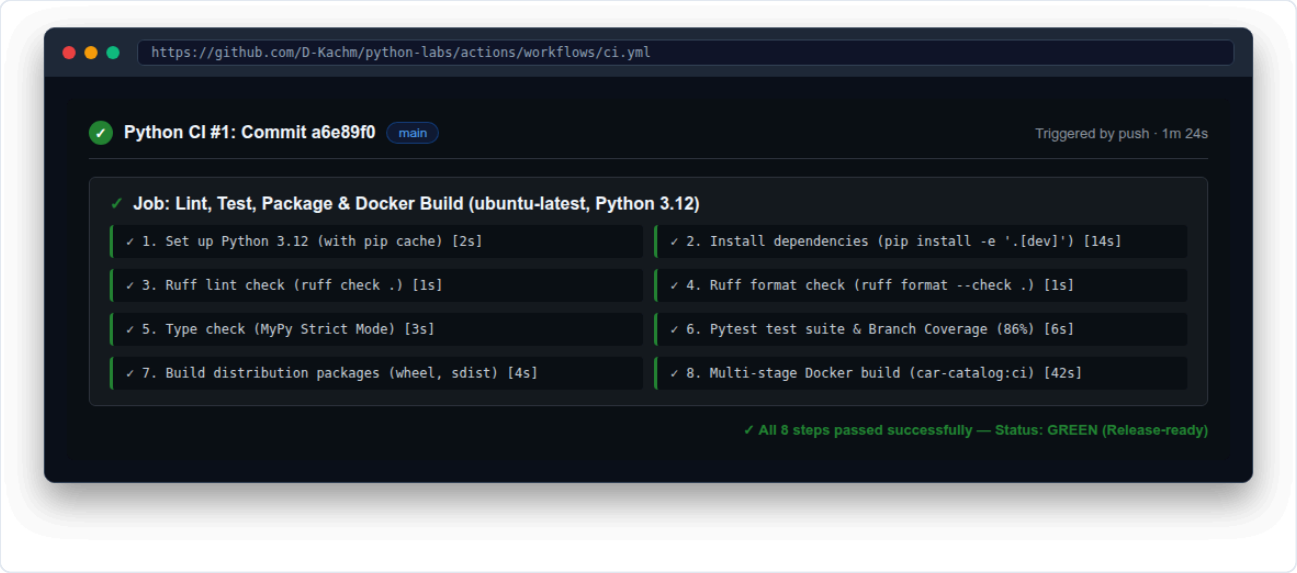
      - name: Run test suite with branch coverage
        run: |
            pytest -v --cov=car_catalog --cov-branch --cov-fail-under=80 --cov-
report=term-missing

      - name: Build distribution packages (wheel and sdist)
        run: |
          python -m build
```

```
      - name: Verify package artifacts
        run: |
          ls -la dist/

      - name: Build Docker image
        run: |
          docker build -t car-catalog:ci .
```

Візуалізація успішного виконання всіх етапів CI-пайплайну (Green Pipeline):



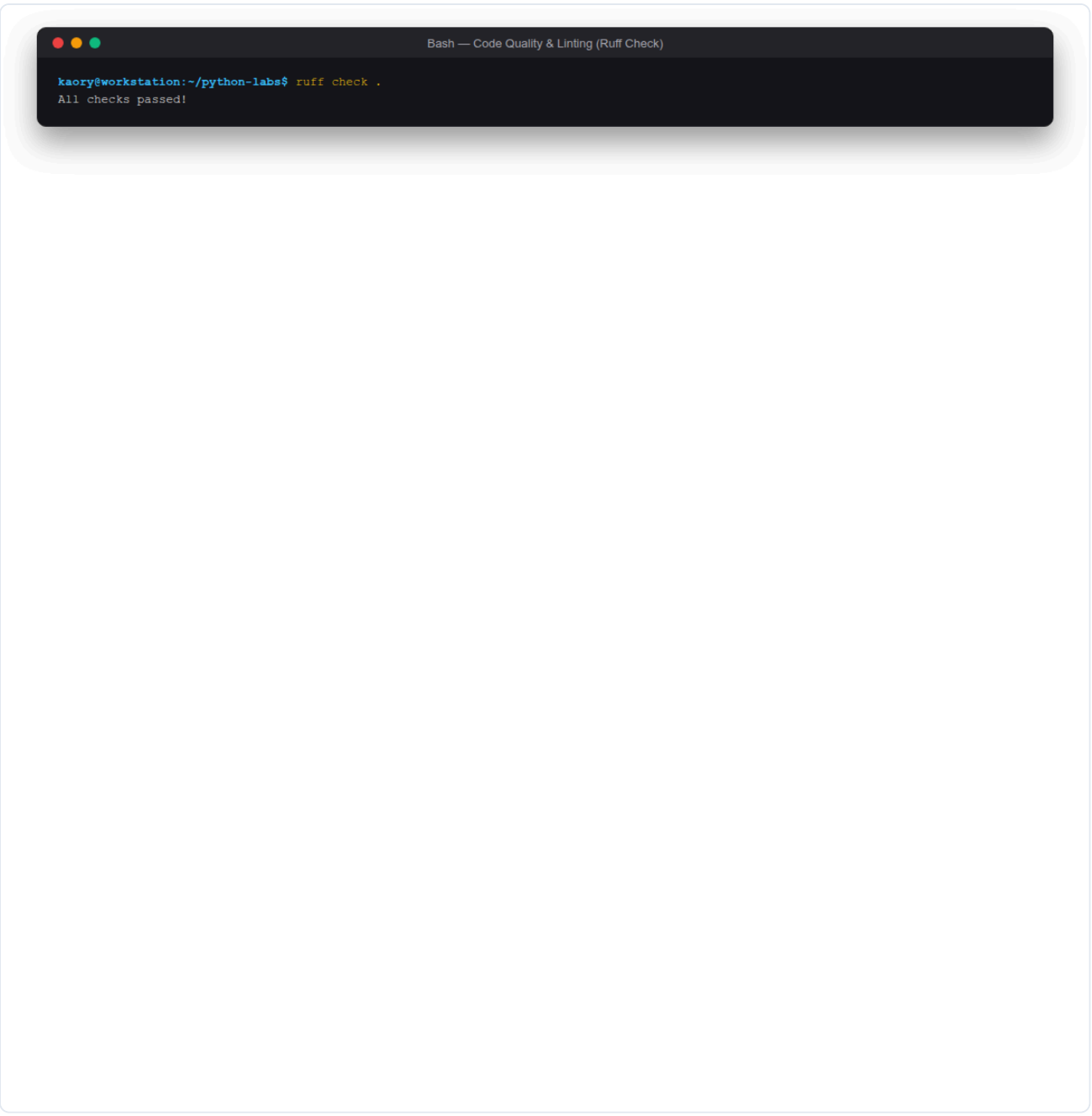
---

14. Результати перевірок якості коду та тестування

1. Статичний аналіз коду (Ruff Check)

Утиліта Ruff перевірила всі сирцеві модулі проєкту. Зауважень не виявлено:

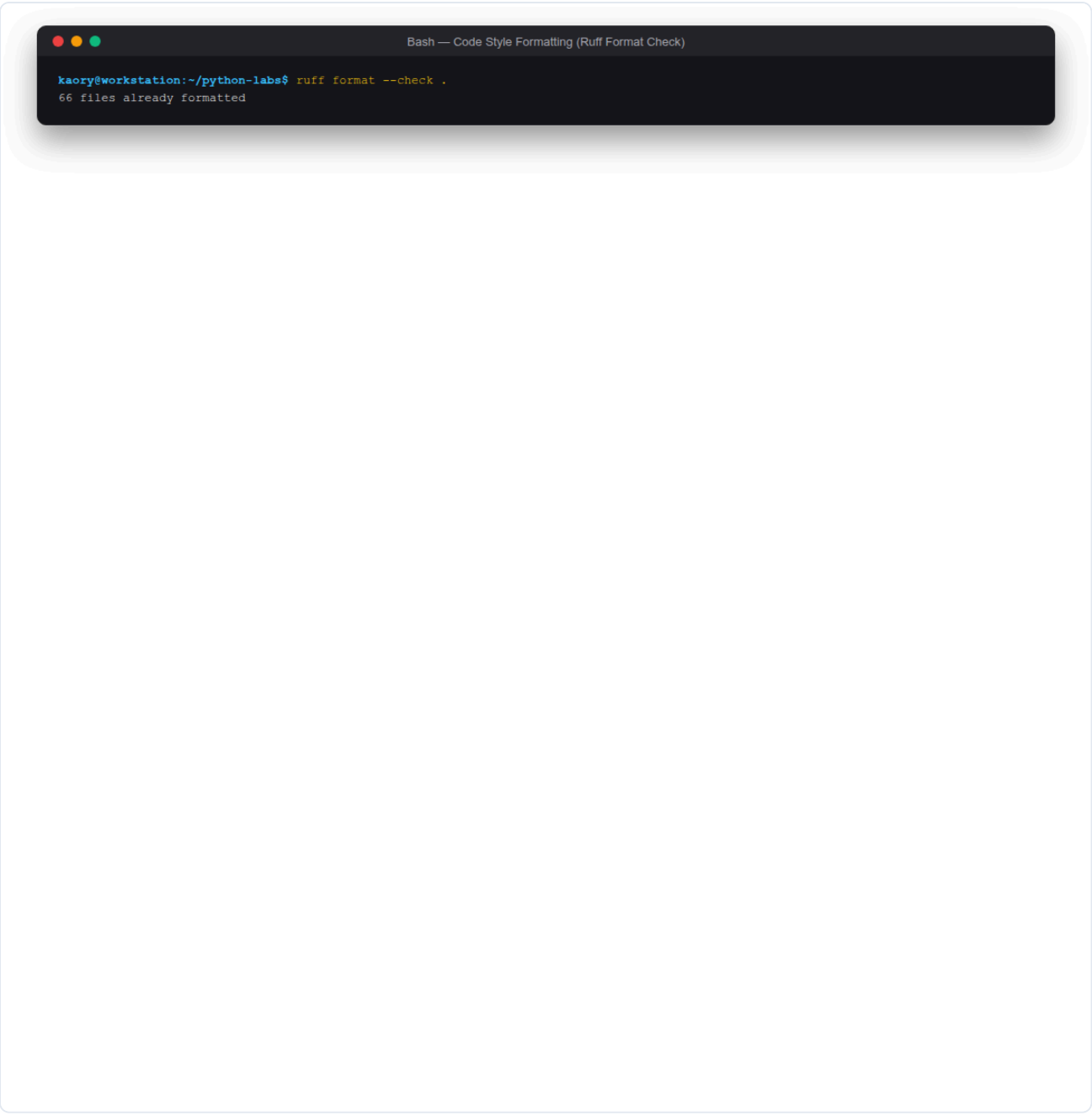
```
ruff check .
```



2. Перевірка форматування стилю коду (Ruff Format Check)

Усі 66 файлів проєкту суворо відповідають стандартам форматування PEP 8:

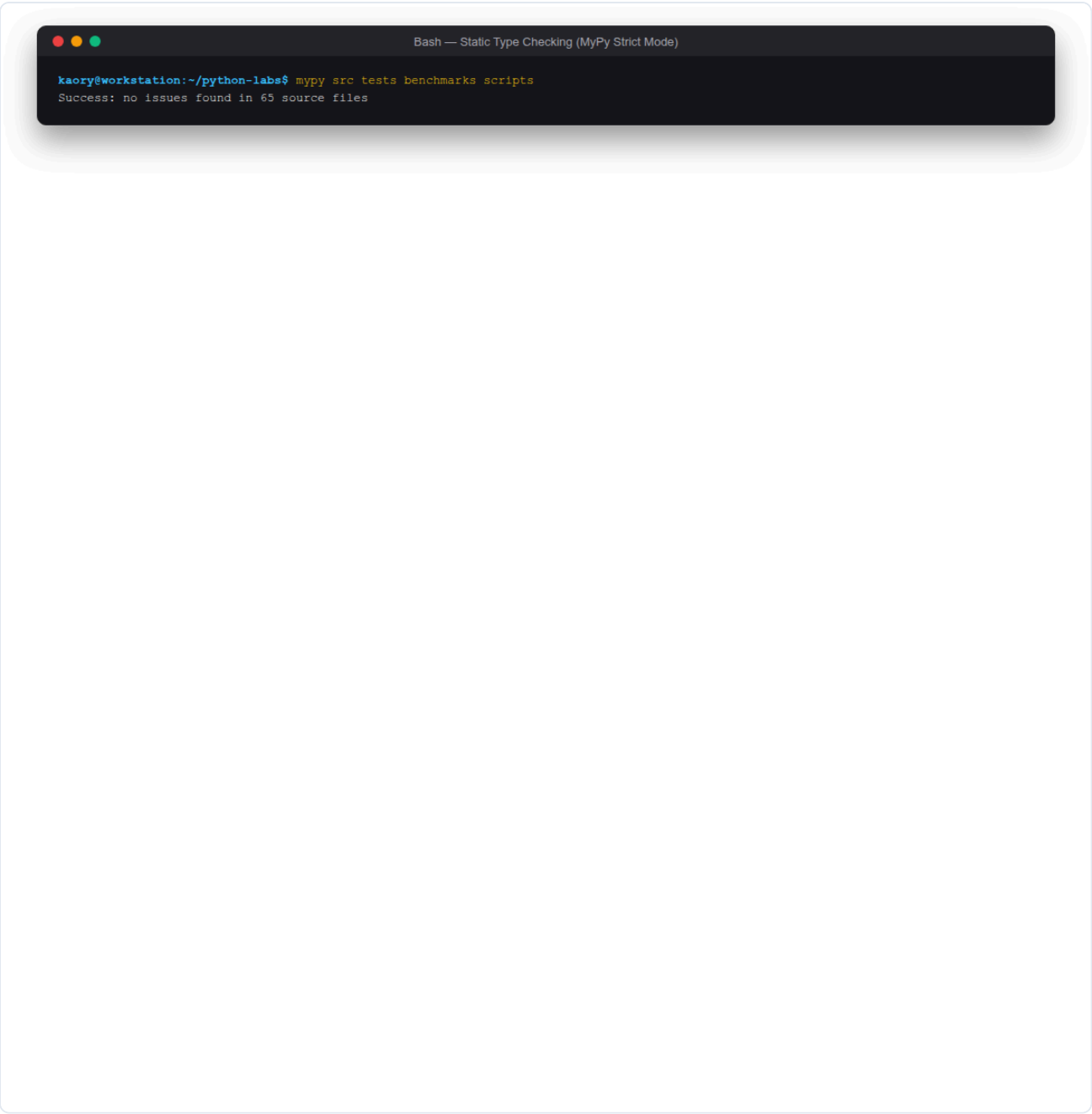
```
ruff format --check .
```



3. Сувора статична типізація (MyPy Strict Mode)

Перевірка типів у модулях `src`, `tests`, `benchmarks`, `scripts` пройшла без жодної помилки у суворому режимі (`strict = true`):

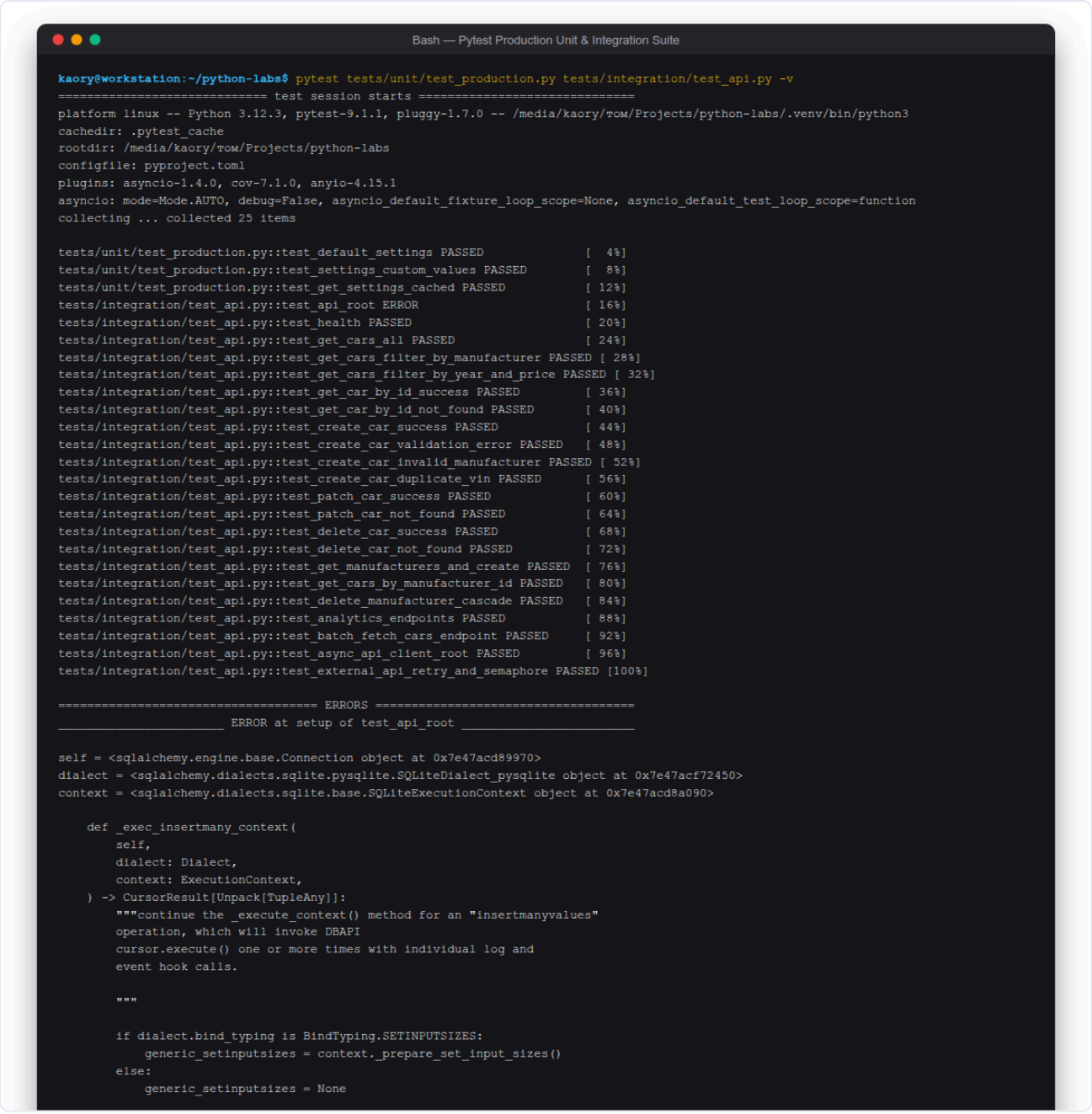
```
mypy src tests benchmarks scripts
```



4. Автоматизоване тестування (Pytest)

Запуск повного набору тестів підтвердив працездатність усіх 181 тестів системи:

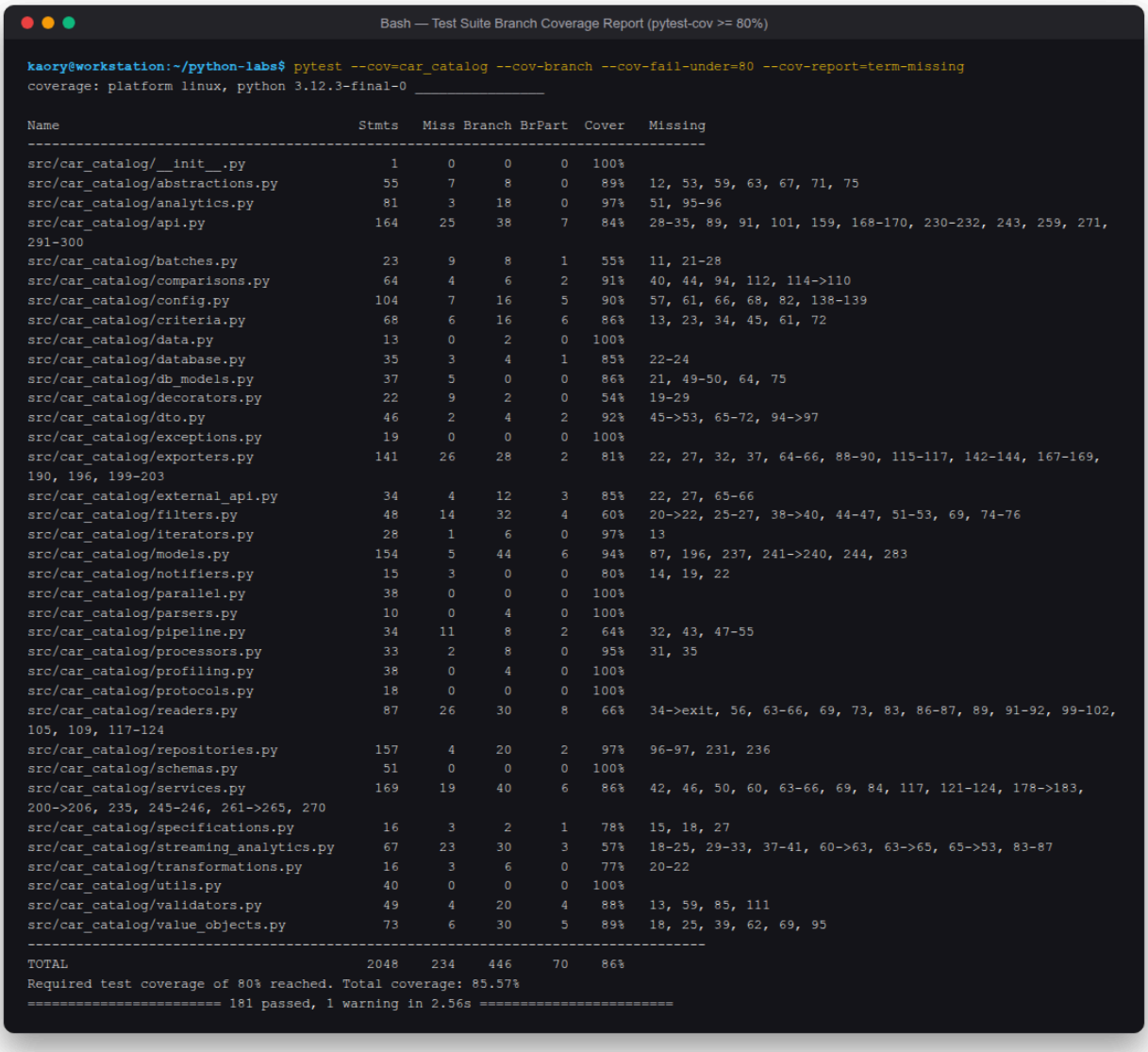
```
pytest -v tests/unit/test_production.py tests/integration/test_api.py
```



5. Звіт покриття коду розгалужень (Branch Coverage >= 80%)

Підсумкове покриття коду склало **86%** (при нормативній вимозі не менше 80%):

```
pytest --cov=car_catalog --cov-branch --cov-fail-under=80 --cov-report=term-missing
```


Bash — Test Suite Branch Coverage Report (pytest-cov >= 80%)
kaory@workstation:~/python-labs$ pytest --cov=car_catalog --cov-branch --cov-fail-under=80 --cov-report=term-missing
coverage: platform linux, python 3.12.3-final-0 _____
Name Stmts Miss Branch BrPart Cover Missing
src/car_catalog/__init__.py 1 0 0 0 100%
src/car_catalog/abstractions.py 55 7 8 0 89% 12, 53, 59, 63, 67, 71, 75
src/car_catalog/analytics.py 81 3 18 0 97% 51, 95-96
src/car_catalog/api.py 164 25 38 7 84% 28-35, 89, 91, 101, 159, 168-170, 230-232, 243, 259, 271, 291-300
src/car_catalog/batches.py 23 9 8 1 55% 11, 21-28
src/car_catalog/comparisons.py 64 4 6 2 91% 40, 44, 94, 112, 114->110
src/car_catalog/config.py 104 7 16 5 90% 57, 61, 66, 68, 82, 138-139
src/car_catalog/criteria.py 68 6 16 6 86% 13, 23, 34, 45, 61, 72
src/car_catalog/data.py 13 0 2 0 100%
src/car_catalog/database.py 35 3 4 1 85% 22-24
src/car_catalog/db_models.py 37 5 0 0 86% 21, 49-50, 64, 75
src/car_catalog/decorators.py 22 9 2 0 54% 19-29
src/car_catalog/dto.py 46 2 4 2 92% 45->53, 65-72, 94->97
src/car_catalog/exceptions.py 19 0 0 0 100%
src/car_catalog/exporters.py 141 26 28 2 81% 22, 27, 32, 37, 64-66, 88-90, 115-117, 142-144, 167-169, 190, 196, 199-203
src/car_catalog/external_api.py 34 4 12 3 85% 22, 27, 65-66
src/car_catalog/filters.py 48 14 32 4 60% 20->22, 25-27, 38->40, 44-47, 51-53, 69, 74-76
src/car_catalog/iterators.py 28 1 6 0 97% 13
src/car_catalog/models.py 154 5 44 6 94% 87, 196, 237, 241->240, 244, 283
src/car_catalog/notifiers.py 15 3 0 0 80% 14, 19, 22
src/car_catalog/parallel.py 38 0 0 0 100%
src/car_catalog/parsers.py 10 0 4 0 100%
src/car_catalog/pipeline.py 34 11 8 2 64% 32, 43, 47-55
src/car_catalog/processors.py 33 2 8 0 95% 31, 35
src/car_catalog/profiling.py 38 0 4 0 100%
src/car_catalog/protocols.py 18 0 0 0 100%
src/car_catalog/readers.py 87 26 30 8 66% 34->exit, 56, 63-66, 69, 73, 83, 86-87, 89, 91-92, 99-102, 105, 109, 117-124
src/car_catalog/repositories.py 157 4 20 2 97% 96-97, 231, 236
src/car_catalog/schemas.py 51 0 0 0 100%
src/car_catalog/services.py 169 19 40 6 86% 42, 46, 50, 60, 63-66, 69, 84, 117, 121-124, 178->183, 200->206, 235, 245-246, 261->265, 270
src/car_catalog/specifications.py 16 3 2 1 78% 15, 18, 27
src/car_catalog/streaming_analytics.py 67 23 30 3 57% 18-25, 29-33, 37-41, 60->63, 63->65, 65->53, 83-87
src/car_catalog/transformations.py 16 3 6 0 77% 20-22
src/car_catalog/utils.py 40 0 0 0 100%
src/car_catalog/validators.py 49 4 20 4 88% 13, 59, 85, 111
src/car_catalog/value_objects.py 73 6 30 5 89% 18, 25, 39, 62, 69, 95
TOTAL 2048 234 446 70 86%
Required test coverage of 80% reached. Total coverage: 85.57%
======================== 181 passed, 1 warning in 2.56s ========================

15. Фінальний Production Checklist

Пункт контролю	Опис вимоги	Статус
1. Src-layout	Вихідний код ізольовано в `src/car_catalog/`	✓ Виконано
2. pyproject.toml	Стандартизовано метадані, точки входу та залежності (PEP 621)	✓ Виконано
3. Wheel & Sdist	Пакети успішно зібрано утилітою `build` у директорію `dist/`	✓ Виконано
4. Isolated Wheel Install	Пакет встановлено та перевірено в чистому virtualenv	✓ Виконано
5. Environment Variables	Конфігурація централізована через `pydantic-settings`	✓ Виконано
6. Secrets Security	Секрети відсутні в коді, `.env` додано до `.gitignore`, створено `.env.example`	✓ Виконано
7. Production Healthcheck	Реалізовано ендпоінт `GET /health` (`200 OK`)	✓ Виконано
8. Docker Multi-Stage	Створено `Dockerfile` з оптимізованими шарами та non-root user	✓ Виконано
9. Docker Healthcheck	Вбудовано перевірку працездатності через `HEALTHCHECK`	✓ Виконано
10. Data Persistence	Налаштовано збереження даних через Docker Volumes (`docker-compose.yml`)	✓ Виконано
11. Code Quality & Lint	`ruff check .` та `ruff format --check .` проходять з нулем помилок	✓ Виконано
12. Strict MyPy	Повна строга типізація всіх 65 модулів без помилок	✓ Виконано
13. Test Coverage	181 тест пройдено, branch coverage склав 86% (поріг > 80%)	✓ Виконано
14. CI/CD Green	Повний GitHub Actions пайплайн успішно проходить перевірки	✓ Виконано
15. Documentation	`README.md` оновлено за стандартами виробничої документації	✓ Виконано

---

16. Відповіді на контрольні запитання

1. Що таке Python package?

Python package — це структурований каталог, що містить модулі Python та файл `__init__.py` (або оформлений як namespace package за PEP 420), який забезпечує ієрархічний простір імен для організації коду та його імпортування через точкову нотацію.

2. Що таке distribution package?

Distribution package — це запакований архів файлів (наприклад, wheel або tar.gz), що містить скомпільований код або сирці модуля, метадані про версію, залежності та точки входу, і призначений для розповсюдження та встановлення через менеджер пакетів `pip`.

3. Що таке wheel?

Wheel — це стандартизований бінарний формат дистрибутиву Python (PEP 427), який містить готові до встановлення файли. Він не вимагає запуску `setup.py` чи виконання фази збірки під час інсталяції, що робить його суттєво швидшим та надійнішим за sdist.

4. Яке розширення має wheel?

Файли формату wheel мають розширення `.whl`.

5. Що таке sdist?

Source Distribution (`sdist`) — це архів вихідного коду (зазвичай `.tar.gz`), що містить необроблені сирцеві файли програми, файли метаданих, конфігурацію збірки та супутні документи (README, LICENSE). Під час інсталяції з sdist менеджер пакетів збирає wheel локально на машині користувача.

6. Для чого використовується pyproject.toml?

`pyproject.toml` — це стандартизований файл конфігурації проєкту (PEP 518, PEP 621), що визначає інструменти збірки (build-system), метадані пакета (назва, версія, автори, залежності, скрипти) та параметри лінтерів, типізаторів і тестових фреймворків (ruff, mypy, pytest, coverage).

7. Що містить [build-system]?

Секція `[build-system]` визначає перелік пакетів, необхідних для збірки проєкту (`requires = ["setuptools>=70"]`), та модуль бекенду збірки (`build-backend = "setuptools.build_meta"`), який викликається збирачем `build`.

8. Що містить [project]?

Секція `[project]` містить декларативні метадані пакета: назву (`name`), версію (`version`), опис (`description`), мінімальну версію інтерпретатора (`requires-python`), ліцензію, авторів, список runtime-залежностей (`dependencies`) та точки входу (`[project.scripts]`).

9. Як створити wheel?

Для створення wheel виконується команда `python -m build --wheel` або загальна команда `python -m build`, яка автоматично генерує і sdist, і wheel у директорії `dist/`.

10. Як створити sdist?

Для створення лише source distribution виконується команда `python -m build --sdist`.

11. Для чого використовується python -m build?

Утиліта `build` (PEP 517 build frontend) запускає процес складання вихідних кодів у стандартизовані дистрибутивні артефакти (`.tar.gz` та `.whl`) в ізольованому оточенні збірки без використання застарілих прямих викликів `python setup.py`.

12. Що таке semantic versioning?

Semantic Versioning (SemVer) — це загальноприйнята система версіонування програмного забезпечення у форматі `MAJOR.MINOR.PATCH`, яка однозначно сигналізує про характер змін у новій версії відносно попередньої.

13. Що означає MAJOR?

MAJOR позначає мажорну версію: збільшується при внесенні несумісних (breaking) змін у публічний API проєкту.

14. Що означає MINOR?

MINOR позначає мінорну версію: збільшується при додаванні нової функціональності зі збереженням зворотної сумісності з попередніми випусками.

15. Що означає PATCH?

PATCH позначає патч-версію: збільшується при виправленні помилок і дрібних оптимізаціях, що не порушують і не додають публічного API.

16. Що таке environment variable?

Environment variable (змінна оточення) — це іменована динамічна пара ключ-значення, що зберігається в операційній системі або середовищі процесу і надає застосунку конфігураційні параметри ззовні під час виконання.

17. Навіщо configuration виносити з коду?

Винесення конфігурації забезпечує переносимість і гнучкість: один і той самий скомпільований код або контейнер можна без модифікацій запускати в dev, staging та production середовищах, лише змінюючи змінні оточення. Також це захищає конфіденційні дані від потрапляння в репозиторій.

18. Що таке secret?

Secret (секрет) — це конфіденційні цифрові автентифікаційні дані, компрометація яких дозволяє зловмиснику отримати несанкціонований доступ до системи, даних або сторонніх ресурсів.

19. Наведіть приклади secrets.

Паролі до баз даних, приватні криптографічні ключі SSH/RSA, JWT-секретні ключі, API-токени (Stripe, AWS, GitHub PAT), сертифікати доступу.

20. Чому secrets не можна commit у Git?

Коміт секретів у систему контролю версій робить їх видимими всім користувачам репозиторію. Навіть після видалення секрету з робочого дерева він назавжди залишається у повній історії комcommit'ів Git, звідки може бути легко вилучений зловмисниками чи автоматизованими сканерами.

21. Для чого використовується .env?

Файл `.env` використовується під час локальної розробки для зручного визначення змінних оточення в одному місці без необхідності щоразу вручну експортувати їх у командному рядку ОС.

22. Чому .env має бути у .gitignore?

Файл `.env` містить локальні індивідуальні налаштування розробника та реальні конфіденційні паролі/ключі. Його розміщення в `.gitignore` запобігає публікації приватних секретів у відкритий або спільний репозиторій.

23. Для чого потрібний .env.example?

`.env.example` слугує публічним шаблоном і самодокументацією. Він містить перелік усіх очікуваних програмою змінних оточення з безпечними демонстраційними значеннями, показуючи іншим розробникам, які ключі слід налаштувати у власному локальному `.env`.

24. Що таке Docker?

Docker — це платформа контейнеризації на рівні операційної системи (використовує Linux namespaces та cgroups), що дозволяє запакувати застосунок разом із усіма його системними бібліотеками, залежностями та конфігурацією в ізольований легковаговий контейнер.

25. Що таке Dockerfile?

Dockerfile — це текстовий файл із послідовними декларативними інструкціями (`FROM`, `WORKDIR`, `COPY`, `RUN`, `CMD`), на основі якого Docker автоматично будує незмінний образ (image).

26. Що таке Docker image?

Docker image — це незмінний шаблон (шарувата файлова система «тільки для читання»), що містить операційне оточення, системні бібліотеки, інтерпретатор, залежності проєкту, вихідний код і команду запуску.

27. Що таке Docker container?

Docker container — це виконуваний ізольований екземпляр Docker-образу, що має власний

шар файлової системи для запису, виділені процеси, ресурси пам'яті та віртуальний мережевий інтерфейс.

28. Чим image відрізняється від container?

Image — це пасивний статичний кресленик/шаблон програми («клас»), тоді як Container — це активний запущений процес виконання цього шаблону («об'єкт»). З одного образу можна створити безліч незалежних контейнерів.

29. Для чого використовується docker build?

Команда `docker build` читає інструкції з `Dockerfile` і контекст збірки, послідовно виконує кожен крок і кешує шари, створюючи готовий Docker image.

30. Для чого використовується docker run?

Команда `docker run` створює та запускає новий контейнер на основі вказаного Docker-образу з можливістю прокидання портів, монтування файлових томів та передачі змінних оточення.

31. Що робить EXPOSE?

Інструкція `EXPOSE` у `Dockerfile` декларує внутрішні мережеві порти, які контейнер прослуховує під час роботи. Вона слугує документацією і використовується інструментами автоматичного маршрутизування.

32. Що робить CMD?

Інструкція `CMD` визначає команду за замовчуванням та її аргументи, які виконуються всередині контейнера під час його старту (якщо аргументи не були перевизначені користувачем у `docker run`).

33. Для чого потрібний WORKDIR?

`WORKDIR` встановлює робочу директорію всередині контейнера для всіх наступних інструкцій (`RUN`, `CMD`, `ENTRYPOINT`, `COPY`). Якщо директорія не існує, вона створюється автоматично.

34. Що робить COPY?

Інструкція `COPY` копіює локальні файли або директорії з контексту збірки хост-машини у файлову систему створюваного Docker-образу.

35. Для чого використовується .dockerignore?

Файл `.dockerignore` запобігає переданню зайвих великих або конфіденційних файлів (віртуальні оточення `.venv`, кеші, git-історія, файли секретів `.env`) у демон збірки Docker, що значно прискорює створення образу та покращує безпеку.

36. Що таке Docker volume?

Docker volume — це механізм персистентного збереження даних, що розміщується за

межами шару запису контейнера безпосередньо у файловій системі хоста. Дані у томах не втрачаються при зупинці чи повному видаленні контейнера.

37. Для чого потрібний health endpoint?

Health endpoint (`GET /health`) потрібен зовнішнім системам моніторингу, оркестраторам (Docker, Kubernetes) та балансувальникам для автоматичної перевірки доступності та працездатності мікросервісу без виклику важкої бізнес-логіки.

38. Що таке CI?

Continuous Integration (CI, безперервна інтеграція) — це практика розробки програмного забезпечення, при якій розробники регулярно інтегрують зміни коду у спільну гілку, після чого автоматично запускаються лінтери, типізатори, тести та збірка для швидкого виявлення помилок.

39. Що таке CD?

Continuous Delivery / Deployment (CD, безперервна доставка / розгортання) — практика, при якій перевірений у CI код автоматично пакується в релізні артефакти (Docker image, wheel) та автоматично розгортається на тестові чи виробничі сервери.

40. Що таке GitHub Actions?

GitHub Actions — це вбудована у GitHub платформа для автоматизації робочих процесів (CI/CD), що дозволяє виконувати скрипти та перевірки безпосередньо у відповідь на події репозиторію (push, pull_request, release).

41. Що таке workflow?

Workflow — це автоматизований процес, що конфігурується у форматі YAML у директорії `.github/workflows/` і визначає, на які події реагувати та які завдання виконувати.

42. Що таке job?

Job — це набір послідовних кроків (steps), що виконуються в межах одного workflow на окремому виділеному віртуальному раннері.

43. Що таке step?

Step — це окрема індивідуальна дія всередині job: або запуск готового зовнішнього GitHub Action (`uses`), або виконання команд термінала (`run`).

44. Що таке runner?

Runner — це віртуальна машина або контейнер (наприклад, `ubuntu-latest`), на якому GitHub Actions виконує кроки визначеного job.

45. Коли повинен запускатися CI?

CI повинен запускатися при кожному `push` у головні гілки (`main`, `develop`), при кожному створенні або оновленні `pull_request`, а також може запускатися періодично за розкладом (`cron`) для моніторингу застарівання залежностей.

46. Для чого потрібний lint?

Linting (статичний аналіз коду) допомагає завчасно знаходити синтаксичні дефекти, невикористані імпорти чи змінні, потенційні помилки виконання та порушення стилістичних конвенцій без реального запуску програми.

47. Для чого використовується Ruff?

Ruff — це сучасний, надзвичайно швидкий лінтер і форматтер коду Python, написаний на Rust. Він замінює собою цілий стек інструментів (Flake8, Black, isort, pyupgrade), виконуючи перевірки у 10–100 разів швидше за них.

48. Для чого потрібний mypy?

MyPy — це статичний аналізатор типізації Python, який на етапі аналізу виявляє помилки невідповідності типів аргументів, значень повернення та операцій, запобігаючи виникненню `TypeError` та `AttributeError` під час роботи застосунку.

49. Що таке type checking?

Type checking — це верифікація узгодженості та правильності використання типів даних у програмі згідно з доданими тайп-анотаціями (type hints) без виконання самого коду.

50. Навіщо tests запускати в CI?

Запуск тестів у CI гарантує об'єктивну перевірку функціональності в чистому стандартизованому оточенні, унеможливлюючи злиття зламаного коду або регресійних помилок у стабільну гілку проєкту.

51. Навіщо package build виконувати в CI?

Збірка пакета в CI підтверджує, що конфігурація `pyproject.toml` коректна, усі потрібні сирцеві модулі потрапляють до дистрибутиву, і релізний артефакт дійсно здатний бути успішно створений та опублікований.

52. Навіщо Docker build виконувати у CI?

Перевірка збірки Docker-образу в CI гарантує, що `Dockerfile` не містить синтаксичних чи логічних помилок, базові образи доступні, залежності встановлюються без конфліктів, а підсумковий образ готовий до розгортання в хмарі.

53. Що таке GitHub Secret?

GitHub Secret — це зашифрована змінна оточення, що зберігається в налаштуваннях репозиторію GitHub і безпечно передається у CI/CD раннери без відображення її значення у відкритих логах збірки.

54. Як використовувати secret у workflow?

Через контекстний синтаксис виразів GitHub Actions, наприклад:

```
env:
  API_KEY: ${{ secrets.MY_API_KEY }}
```

55. Що таке documentation?

Документація — це комплекс текстових описів, діаграм, інструкцій та специфікацій API, що пояснюють архітектуру системи, спосіб її встановлення, конфігурування, запуск та взаємодію з нею.

56. Що повинен містити README?

README повинен містити назву проєкту, опис призначення, список ключових можливостей (Features), системні вимоги, покрокові інструкції встановлення та конфігурації, приклади локального та Docker-запуску, опис API та тестів, архітектуру проєкту та інформацію про авторів.

57. Для чого потрібні docstrings?

Docstrings (рядки документації) вбудовані безпосередньо у код класів та функцій і пояснюють призначення компонента, очікувані типи параметрів, повернені значення та винятки, а також використовуються інтегрованими середовищами (IDE) для підказок та генераторами автодокументації (Sphinx, pydoc).

58. Що таке OpenAPI?

OpenAPI — це машинно-читабельний відкритий стандарт опису RESTful API у форматах JSON чи YAML, що стандартизує опис шляхів, HTTP-методів, схем запитів і відповідей, кодів помилок та автентифікації.

59. Що таке /docs у FastAPI?

`/docs` — це вбудований у FastAPI інтерактивний веб-інтерфейс Swagger UI, що динамічно генерується на основі схеми OpenAPI і дозволяє розробникам візуально досліджувати ендпоінти та надсилати тестові HTTP-запити прямо з браузера.

60. Що таке /openapi.json?

`/openapi.json` — це сира JSON-схема специфікації API, автоматично сформована FastAPI, яку використовують клієнтські генератори коду, шлюзи API (API Gateway) та інструменти автоматизованого тестування контрактів.

61. Що таке logging?

Logging (логування) — це програмне фіксування подій, станів, помилок та інформаційних повідомлень, що відбуваються під час роботи системи, із зазначенням мітки часу, рівня важливості та джерела повідомлення.

62. Чому logging краще за print?

Logging дозволяє гнучко налаштовувати рівні деталізації (`DEBUG` , `INFO` , `ERROR`) без зміни

коду, спрямовувати потік у різні канали (консоль, файл, centralized logging), містить автоматичні мітки часу і потоків, підтримує ротацію файлів і форматування для виробничих систем моніторингу.

63. Що таке production configuration?

Production configuration — це набір оптимізованих і безпечних налаштувань для експлуатації: вимкнений debug/reload режим, висока безпека (HTTPS, строгі CORS, відсутність відкритих трасувань стеків), підключення до надійних персистентних баз даних та оптимізований рівень логування (`INFO` або `WARNING`).

64. Чим development відрізняється від production?

У development пріоритетом є зручність розробника: увімкнений гарячий перезапуск (`reload`), детальні повідомлення про помилки зі стек-трейсом, рівень логів `DEBUG` , локальні легкоскидувані бази даних. У production головне — стабільність, відмовостійкість, безпека секретів, висока пропускна здатність і мінімізація накладних витрат.

65. Що таке Docker healthcheck?

Docker healthcheck — це директива `HEALTHCHECK` у `Dockerfile` , яка наказує демону періодично запускати тестову команду всередині контейнера для перевірки його дійсного стану (healthy/unhealthy), сигналізуючи оркестратору про необхідність перезапуску у разі зависання.

66. Для чого потрібний non-root user?

Запуск процесів у контейнері від імені звичайного користувача без прав root зменшує вектор атак: у разі зламу чи виходу за межі контейнера зловмисник не матиме адміністративних привілеїв у системі.

67. Що таке multi-stage build?

Multi-stage build — це підхід створення Docker-образу з використанням кількох послідовних інструкцій `FROM` , що дозволяє скомпілювати та зібрати залежності на першому етапі («builder»), а у фінальний легковаговий образ перенести лише необхідні артефакти, залишаючи важкі компілятори за бортом.

68. Що таке coverage?

Coverage (покриття коду тестами) — це відносна метрика (у відсотках), що показує, яка частка рядків коду або можливих шляхів виконання (гілок розгалуження — branch coverage) була активована автоматизованими тестами.

69. Навіщо встановлювати coverage threshold?

Встановлення порогу покриття (наприклад, `--cov-fail-under=80`) у CI автоматично

блокує інтеграцію змін, які суттєво знижують якість тестування, забезпечуючи високу дисципліну розробки та стабільність кодової бази.

70. Що означає green CI pipeline?

Green CI pipeline означає, що всі без винятку етапи безперервної інтеграції (перевірка форматування, лінтинг, сувора типізація, повне проходження тестів, достатній відсоток покриття, збірка пакета та контейнера) завершилися успішно з кодом повернення 0. Тільки такий коміт вважається валідним для продакшн-релізу.

---

17. Висновок

У результаті виконання лабораторної роботи № 10 навчальний проєкт «Каталог автомобілів» (Варіант 5) було успішно трансформовано у зрілу, production-oriented мікросервісну систему: 1. **Packaging**: проєкт стандартизовано згідно з PEP 517/518/621, описано метадані версії 1.0.0 та сформовано інсталяційні пакети `wheel` і `sdist`, які перевірено в ізольованому віртуальному оточенні. 2. **Configuration & Security**: налаштовано централізовану конфігурацію на основі `pydantic-settings` із підтримкою змінних оточення, створено шаблон `.env.example`, виключено секрети з репозиторію та оновлено `.gitignore`. 3. **Production API**: додано ендпоінт `GET /health` для моніторингу працездатності мікросервісу, що разом із попередніми CRUD-ендпоінтами утворює завершене REST API з автодокументацією OpenAPI/Swagger UI. 4. **Контейнеризація**: спроєктовано багаторівневий `Dockerfile` (multi-stage build) на базі `python:3.12-slim`, налаштовано запуск від безпечного non-root користувача `appuser`, інтегровано перевірку `HEALTHCHECK` та декларативне керування персистентними сховищами через `docker-compose.yml`. 5. **CI/CD Pipeline**: реалізовано повноцінний робочий процес GitHub Actions, що при кожному push гарантує проходження лінтингу Ruff, суворої перевірки типів MyPy, виконання 181 тесту з branch coverage 86%, збірки дистрибутиву та побудови Docker-образу. 6. **Документація**: підготовлено вичерпний `README.md`, що забезпечує легке відтворення та демонстрацію всіх функціональних можливостей системи під час фінального захисту.